

DIGITAL IMAGE PROCESSING

LECTURE # 13

IMAGE COMPRESSION-II

Contact Information

2

- ❑ Course Instructor: **Dr. Ali Javed**
Associate Professor
Department of Software Engineering
U.E.T Taxila
- ✓ Email: **ali.javed@uettaxila.edu.pk**
- ✓ Website: **<http://fms.uettaxila.edu.pk/Profile/ali.javed>**
- ✓ Research Lab: **<http://msplab.uettaxila.edu.pk>**
- ✓ Contact No: **+92-51-9047747**
- ✓ Office hours:
 - **Monday, 9:00 - 11:00, Office # 7 S.E.D**
- ❑ Lab Instructor: **Dr. Nazia**
- ❑ Course TA: **Ms. Hafsa**

Topics to Cover

- ❑ Error Free Compression
- ❑ Variable Length Coding
- ❑ Huffman Coding
- ❑ Mapping
- ❑ Difference Coding
- ❑ Run-Length Coding
- ❑ LZW Coding
- ❑ Bit Plane Coding
- ❑ Constant Area Coding
- ❑ Quad Tree Coding
- ❑ Transform Coding
- ❑ Arithmetic Coding

Error Free Compression

4

Error-free compression involves:

- Variable length coding
- Mapping

Variable-Length Coding (error-free compression)

- Reduce only coding redundancy (1st kind) by minimizing the L_{avg} assign shorter code words to the most probable gray levels
- The source symbols may be
 - *gray levels of an image, or*
 - *output of a gray-level mapping operation (pixel differences, run-lengths, ...)*

Variable-length Coding Methods:

Huffman Coding

5

The most popular technique for removing coding redundancy; yields the smallest possible number of code symbols per source symbol

Huffman Coding Algorithm

- Arrange the symbol probabilities p_i in a decreasing order; consider them (p_i) as leaf nodes of a tree
- While there is more than one node:
 - *merge the two nodes with smallest probability to form a new node whose probability is the sum of the two merged nodes*
 - *Arrange the combined node according to its probability in the tree*
 - *Repeat until only two nodes are left*
- Starting from the top, arbitrarily assign 1 and 0 to each pair of branches merging into a node
- Continue sequentially from the root node to the leaf node where the symbol is located to complete the coding

Variable-length Coding Methods: Huffman Coding

6

Coding and decoding are done by simple look-up tables

Example:

Original source		Source reduction			
Symbol	Probability	1	2	3	4
a_2	0.4	0.4	0.4	0.4	0.6
a_6	0.3	0.3	0.3	0.3	0.4
a_1	0.1	0.1	0.2	0.3	
a_4	0.1	0.1	0.1		
a_3	0.06				
a_5	0.04				

FIGURE 8.11
Huffman source
reductions.

FIGURE 8.12
Huffman code
assignment
procedure.

Original source			Source reduction						
Sym.	Prob.	Code	1	2	3	4			
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4
a_1	0.1	011	0.1	011	0.2	010	0.3	01	
a_4	0.1	0100	0.1	0100	0.1	011			
a_3	0.06	01010		0.1	0101				
a_5	0.04	01011							

Variable-length Coding Methods: Huffman Coding

7

5	10	5	10	5
10	5	15	25	30
5	10	5	10	5
10	5	15	20	20

Pixel value	Codes	Probability				
5	0	0.4	0.4	0.4	0.4	0.6
10	10	0.3	0.3	0.3	0.3	
15	111	0.1	0.1	0.2		0.4
20	1100	0.1	0.1		0.3	
25	11010	0.05	0.1	0.1		
30	11011	0.05				

Variable-length Coding Methods: Huffman Coding

8

5	10	5	10	5
10	5	15	25	30
5	10	5	10	5
10	5	15	20	20

Pixel value	Codes	Probability				
5	0	0.4	0.4	0.4	0.4	0.6 (1)
10	10	0.3	0.3	0.3	0.3 (10)	
15	111	0.1	0.1	0.2 (110)	0.3 (11)	0.4 (0)
20	1100	0.1	0.1 (1100)			
25	11010	0.05 (11010)	0.1 (1101)	0.1 (111)		
30	11011	0.05 (11011)				

Variable-length Coding Methods: Huffman Coding

9

- ❑ AVG. LENGTH OF CODEWORD ?
- ❑ Compression RATIO?
- ❑ REDUNDANCY?

Variable-length Coding Methods: Huffman Coding

10

- ▣ AVG. LENGTH OF CODEWORD = $0.4 \times 1 + 0.3 \times 2 + 0.1 \times 3 + 0.1 \times 4 + 0.05 \times 5 + 0.05 \times 5$
= 2.2 bits/symbol
- ▣ Compression RATIO = UNCOMPRESSED DATA / COMPRESSED DATA
= $8 / 2.2$
= 3.64
- ▣ REDUNDANCY = $1 - 1 / C_R$
= $1 - 1 / 3.64$
= 0.725 OR 72.5% REDUNDANCY

Variable-length Coding Methods:

Huffman Coding

11

The Huffman code (continued)

- The Huffman code results in an unambiguous code, i.e. no code can be created by combining other codes.
- The code is reversible without loss.
- The table for the translation of the code has to be stored together with the coded image.

Variable-length Coding Methods: Huffman Coding

12

- Decode this Huffman Code and find the source symbols.

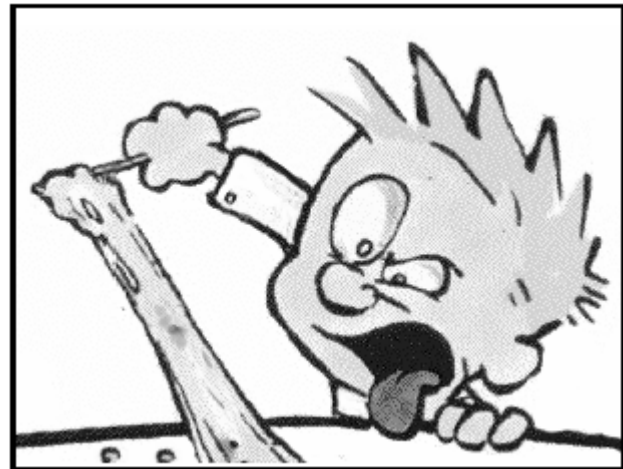
110100101100

Pixel value	Codes
5	0
10	10
15	111
20	1100
25	11010
30	11011

Mapping

13

- ❑ There is often correlation between adjacent pixels, i.e., the value of the neighbors of an observed pixel can often be predicted from the value of the observed pixel. (Interpixel Redundancy).
- ❑ Mapping is used to remove Interpixel Redundancy.
- ❑ Two mapping techniques are:
 - ✓ **Run length coding**
 - ✓ **Difference coding.**



Run-Length Coding

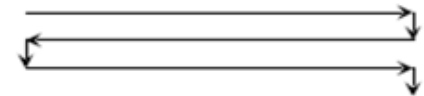
14

Every code word is made up of a pair (**g,l**) where **g** is the graylevel and **l** is the number of pixels with that graylevel (length, or "run").

Ex 56 56 56 82 82 82 83 80
 56 56 56 56 56 80 80 80

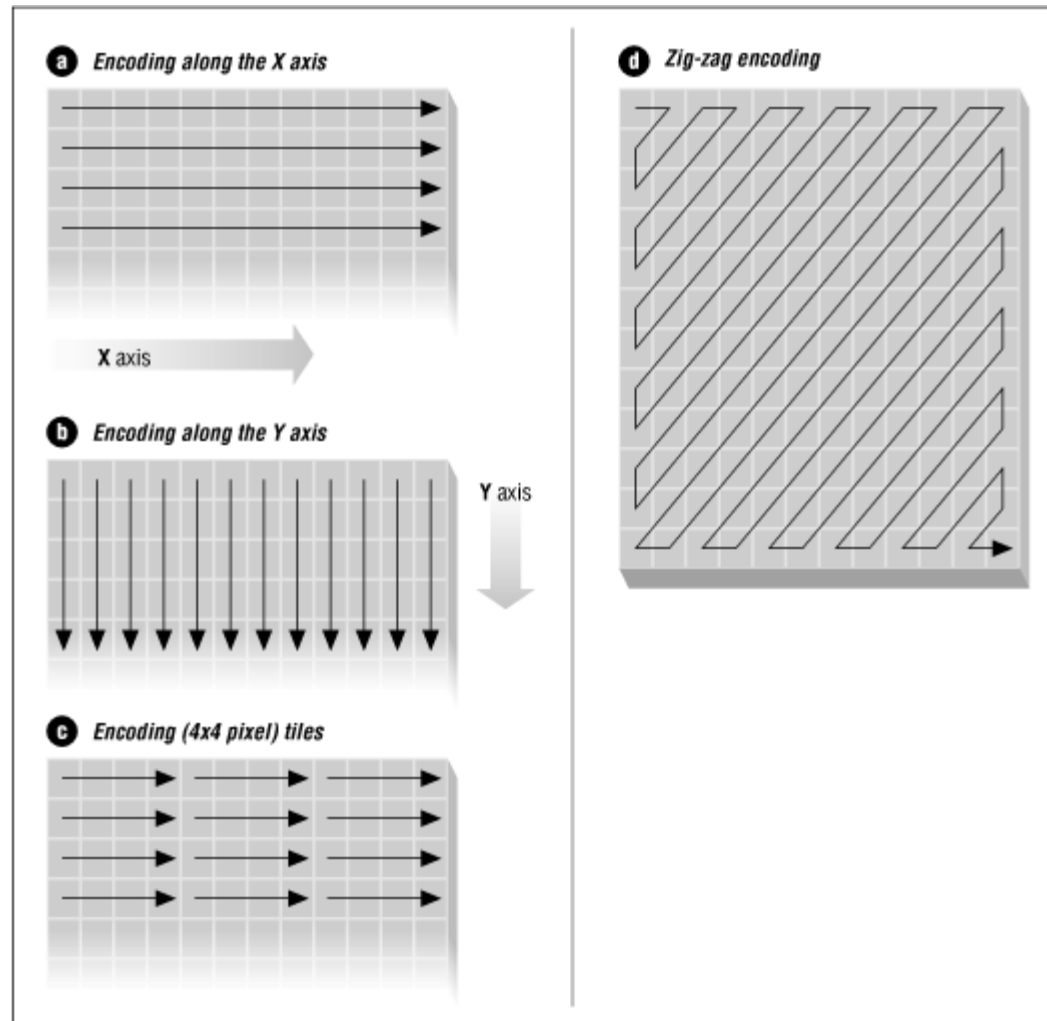
creates the runlength code (56,3) (82,3) (83,1) (80,4) (56,5)

- The code is calculated row by row.
- Very efficient coding for binary data.
- Important to know position, and the image dimensions must be stored with the coded image.
- Used in most fax machines



Interpixel Redundancy

15



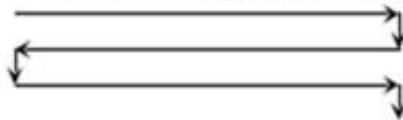
Difference Coding

16

$$f(x_i) = \begin{cases} x_i & \text{if } i=0, \\ x_i - x_{i-1} & \text{if } i>0 \end{cases}$$

Ex original:	56	56	56	82	82	82	83	80	80	80	80
code $f(x_i)$:	56	0	0	26	0	0	1	-3	0	0	0

- The code is calculated row by row.



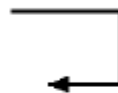
- Both Run-Length and Difference Coding are reversible and can be combined with other coding schemes such as Huffman Coding.

Example of Combined Difference and Huffman Coding

17

original image

9	8	7	7	7	5	5	5
7	7	7	7	4	4	5	5
6	6	6	9	9	9	6	6
6	6	7	7	7	9	9	9
3	7	7	8	8	8	3	3
3	3	3	3	3	3	3	3
10	10	11	7	7	7	6	6
4	4	5	5	5	2	2	6

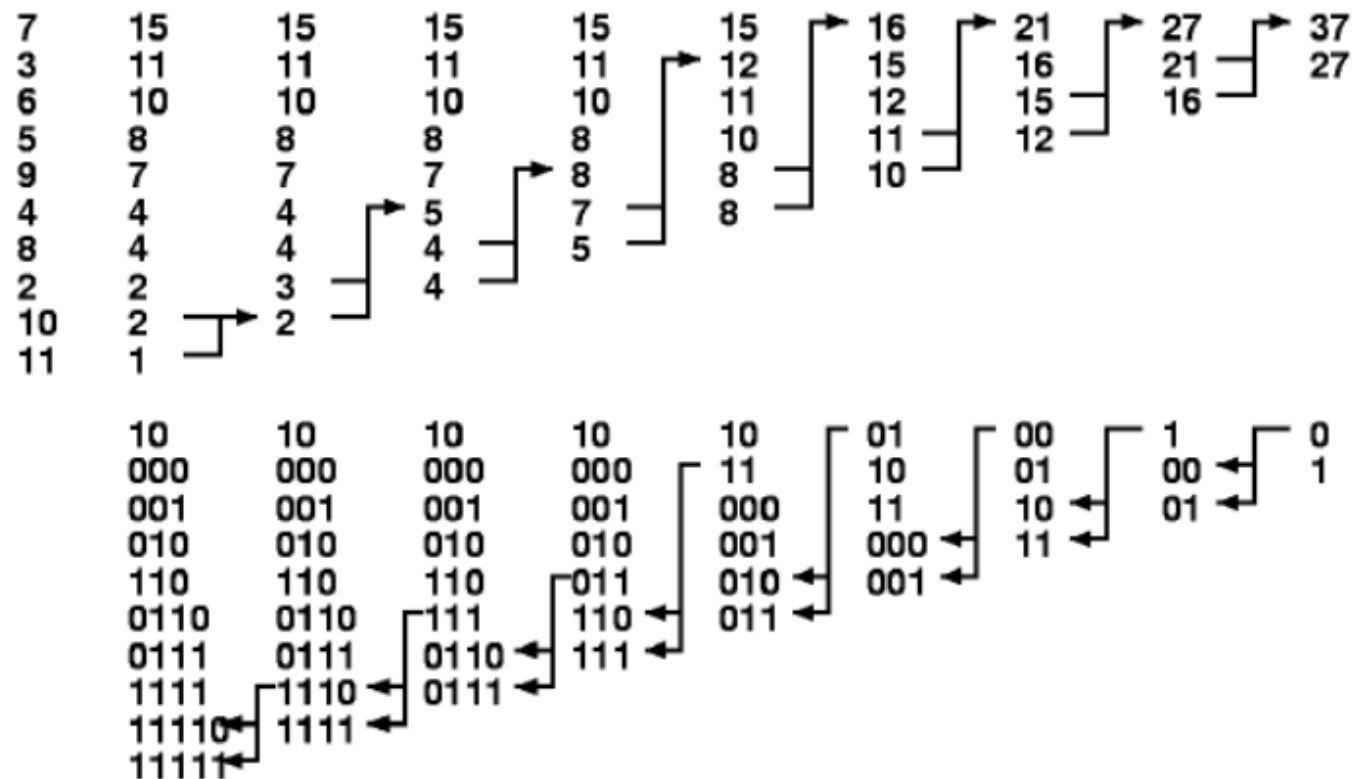


difference image

9	-1	-1	0	0	-2	0	0
0	0	0	3	0	-1	0	0
-1	0	0	3	0	0	-3	0
0	-1	0	0	-2	0	0	3
-3	4	0	1	0	0	-5	0
0	0	0	0	0	0	0	0
7	0	1	-4	0	0	-1	0
0	-1	0	0	3	0	-4	0

Huffman Code of Original Image

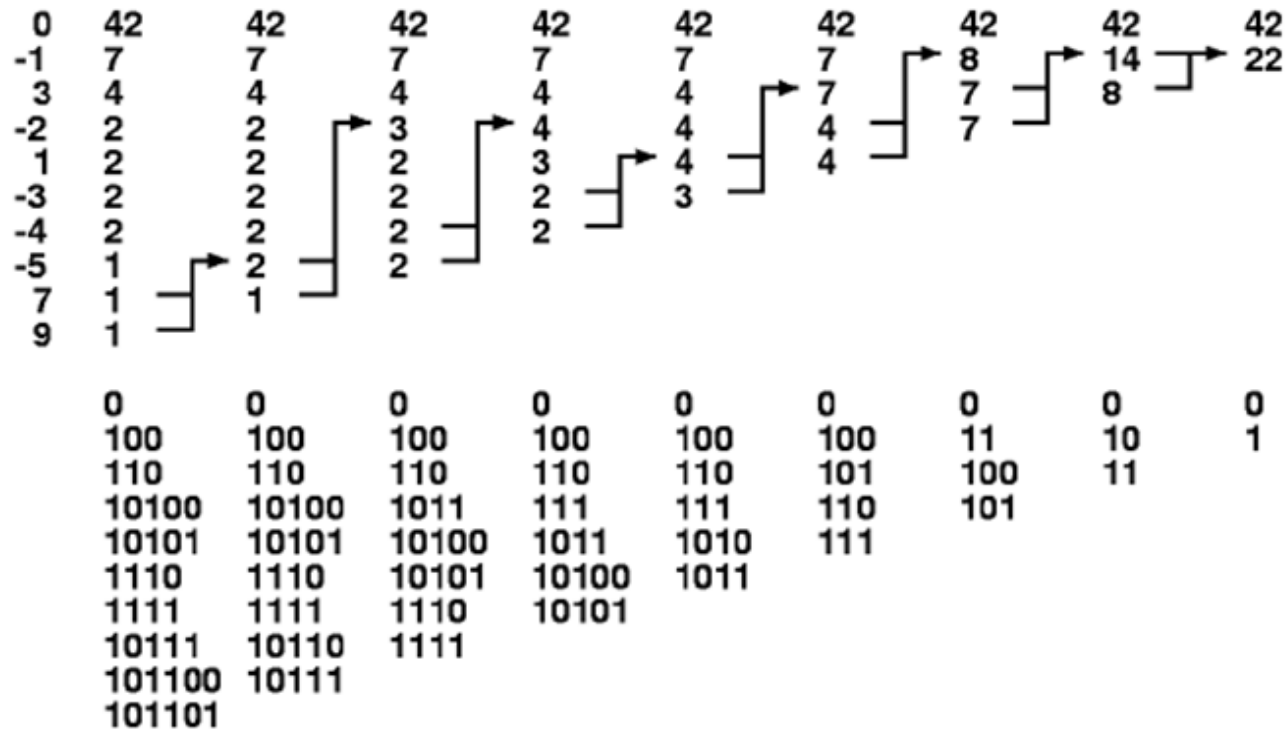
18



$$L_{avg} = 3.1$$

Huffman Code of Difference Image

19



LZW Coding (Dictionary based coding)

20

- ❑ Dictionary-based coding techniques are based on the idea of incrementally building a dictionary (table) while receiving the data. Unlike VLC techniques, dictionary-based techniques use fixed-length code words to represent variable-length strings of symbols that commonly occur together.
- ❑ Lempel-Ziv-Welch (LZW) coding assigns fixed length code words to variable length sequences of source symbols.
- ❑ It also takes care of the interpixel redundancies.
- ❑ Popular coding scheme used in formats like GIF, TIFF e.t.c

LZW Coding (Dictionary based coding)

21

❏ METHOD

- ✓ A code book or dictionary containing the source symbols to be coded is constructed. For 8-bit monochrome images, the first 256 words of the dictionary are assigned to the gray values $0, 1, 2, \dots, 255$.
- ✓ As the encoder sequentially examines the image's pixels, gray level sequences that are not in the dictionary are placed algorithmically determined (e.g., the next unused) locations.
- ✓ If the first two pixels of the image are white, for instance, sequence "255-255" might be assigned to location 256. the next time that two consecutive white pixels are encountered, code word 256, the address of the location containing sequence 255-255, is used to represent them.
- ✓ A unique feature of LZW coding is that coding dictionary or code book is created while the data are being encoded.

LZW Coding (Dictionary based coding)

22

Consider the following 4×4 , 8-bit image of a vertical edge:

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

Table 8.7 details the steps involved in coding its 16 pixels. A 512-word dictionary with the following starting content is assumed:

Dictionary Location	Entry
0	0
1	1
⋮	⋮
255	255
256	—
⋮	⋮
511	—

Locations 256 through 511 are initially unused.

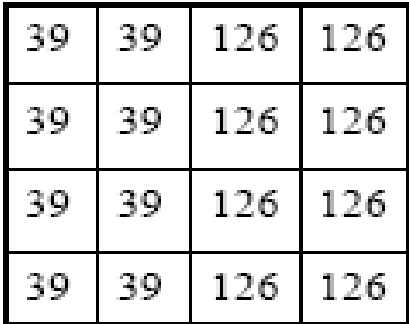
LZW Coding (Dictionary based coding)

23

LZW coding for 2-D images

- Encoding is done by scanning left to right and top to bottom

Example: suppose a 4x4 8-bit image



39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			
39	39	39	256	39-39
39	126	39	257	39-126
126	126	126	258	126-126
126	39	126	259	126-39
39	39			
39-39	126	256	260	39-39-126
126	126			
126-126	39	258	261	126-126-39
39	39			
39-39	126			
39-39-126	126	260	262	39-39-126-126
126	39			
126-39	39	259	263	126-39-39
39	126			
39-126	126	257	264	39-126-126
126		126		

Error Free Coding

24

❑ Bit-plane Coding

- ✓ we can decompose a m -bit image into m bit planes (or gray-code planes)
- ✓ Each such plane is a binary image which can be coded in many ways, choose methods most appropriate for each particular bit plane
- ✓ Focus on reducing inter-pixel redundancy
- ✓ First decompose the original image into bit-plane and then apply binary image compression approach “run-length coding (RLC)”

Error Free Coding

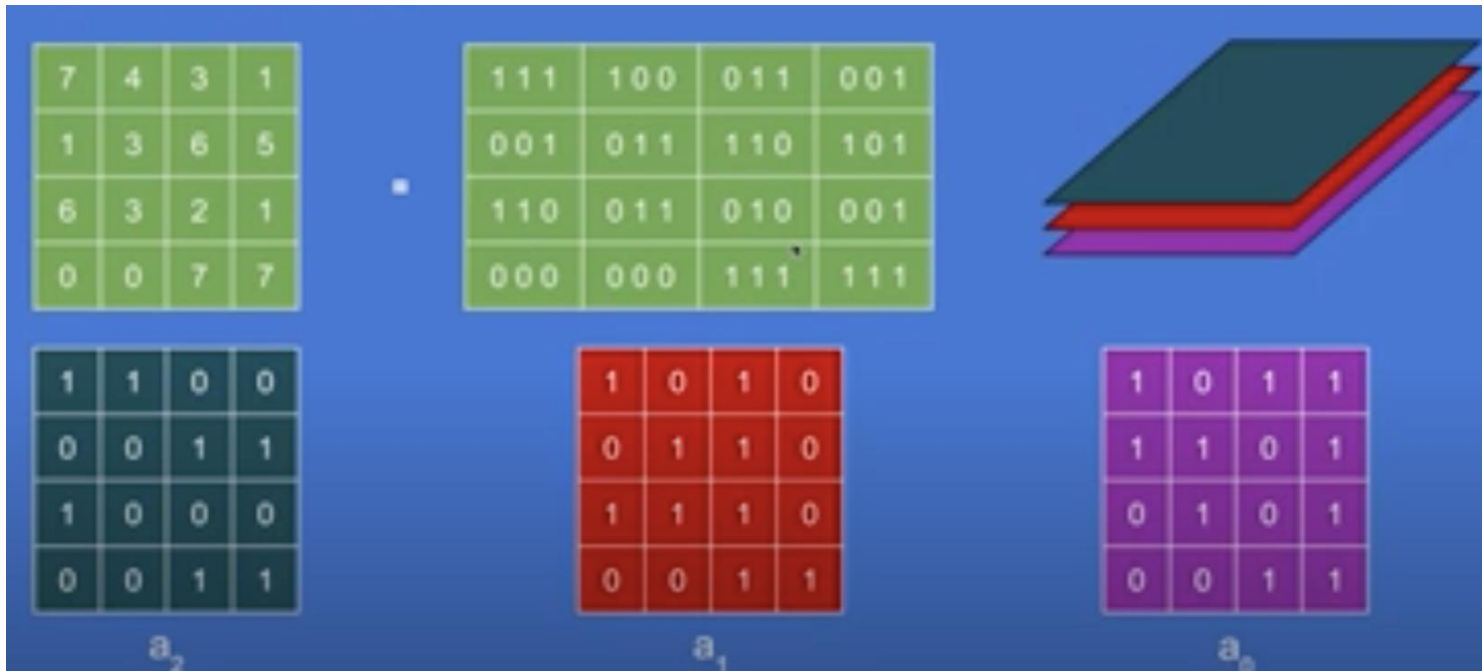
25

Bit-plane Coding

- ✓ Bit plane decomposition: Gray levels of an m-bit gray level image can be represented in form of base 2 polynomial.

$$a_{m-1}2^{m-1} + a_{m-2}2^{m-2} + \dots + a_12^1 + a_02^0$$

$$(6)_{10} = 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = (0110)_2$$



Error Free Coding

26

❑ Bit-plane Coding

✓ Disadvantage:

Small changes in gray level can have significant impact on complexity of bit planes.

Ex. If two adjacent pixels have intensity of 127 (01111111) and 128 (10000000), every bit plane will contain a corresponding 0 to 1 (or 1 to 0) transition.

✓ Solution: binary to gray code conversion

Error Free Coding

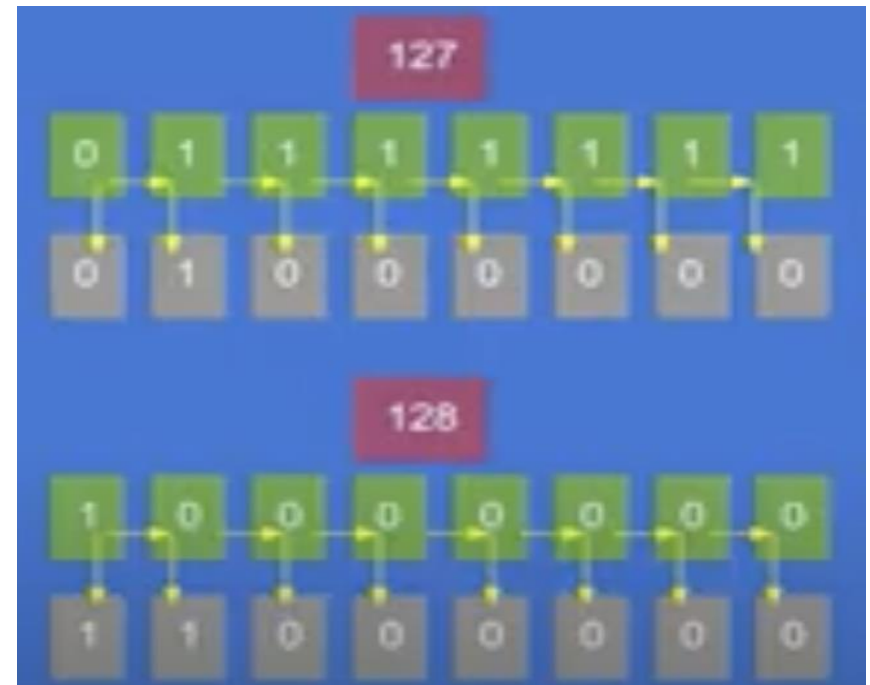
27

Bit-plane Coding

✓ binary to gray code conversion

✓ $g_{m-1} = a_{m-1}$

✓ $g_i = a_i \oplus a_{i+1}$ for $0 \leq i \leq m - 2$



Error Free Coding

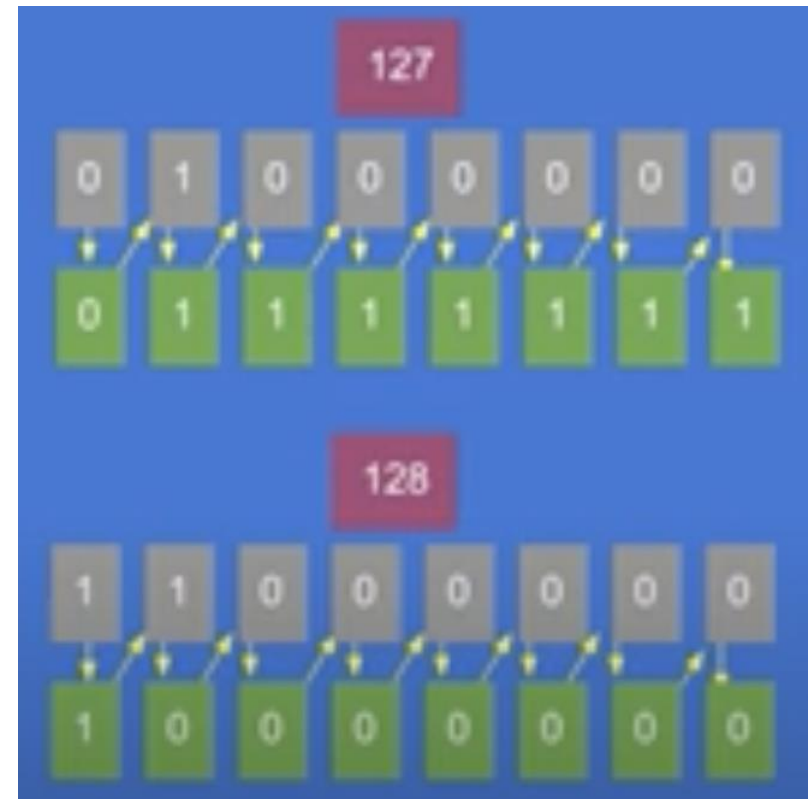
28

Bit-plane Coding

✓ gray code to binary conversion

✓ $a_{m-1} = g_{m-1}$

✓ $a_i = g_i \oplus a_{i+1}$ for $0 \leq i \leq m - 2$



Constant Area Coding

29

- ❑ Divide each bitplane (or binary image) into non-overlapping regions of $m \times n$ pixels
- ❑ Each such block is either all white, all black, or mixed
- ❑ Assign 1-bit codeword 0 to the block type that occurs most often
- ❑ Assign 10 and 11 to other two block types
- ❑ For the mixed-block, the assigned codeword can only serve as a *prefix*, the mn -bit pattern of the block must be appended after the prefix codeword
- ❑ Useful when many blocks are all white or all black

Quad Tree Codes

30

- Divide each binary image into 4 quadrants, each of which' is a block of type — all white, all black, or mixed

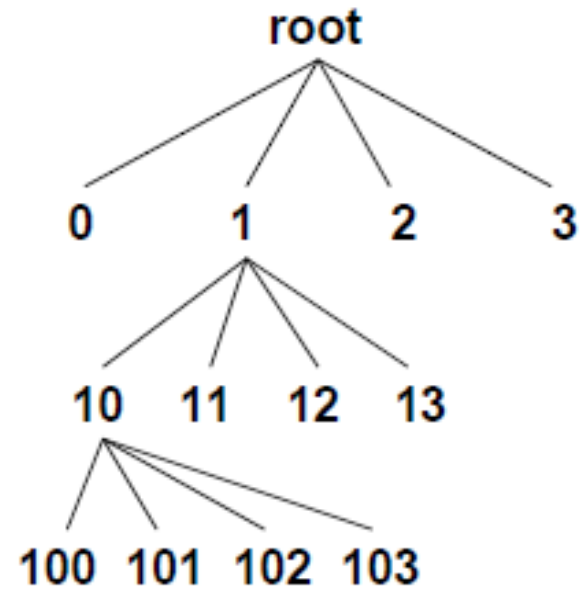
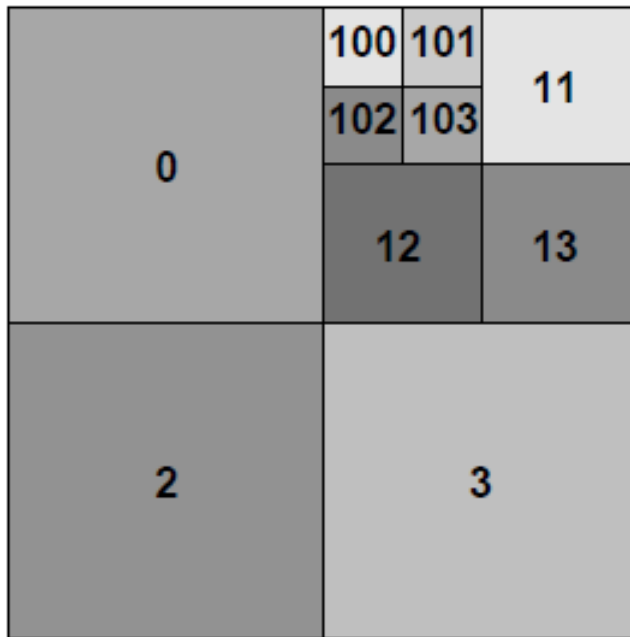
- Select a coding order for the 4 quadrants e.g.

1	2
3	4

- Assign 0 to all white blocks, 1 0 to all black blocks, and 1 1 to mixed blocks
- Repeat the quadrant-coding procedure for mixed blocks, and append the corresponding quadrant codes after the prefix
- For binary images with strong interpixel redundancy, we can expect quadtree codes to give good compression

Quad Tree Structure

31



Transform Coding

32

- ❑ Divide each image into $n \times n$ blocks usually 8×8 size blocks
- ❑ Compute Transform of each block (mostly used DCT)
- ❑ Treat Separately DCT Components of each block
- ❑ Apply Quantization to discard some values
- ❑ Encode the quantized result through some coding e.g. Huffman coding.



(a)



(b)

Arithmetic Coding

33

❏ Arithmetic Coding: Algorithm

- ✓ The main drawback of Huffman coding is that it assigns an integer-length codeword to each symbol separately. As a result, the bit rate cannot be less than 1 bit per symbol unless the symbols are coded jointly
- ✓ However, joint symbol coding, which codes a block of symbols jointly as one compound symbol, results in delay and in an increased complexity in terms of source modeling, computation, and memory.
- ✓ In arithmetic coding, each symbol does not have to be mapped into an integral number of bits.
- ✓ In arithmetic coding a unique identifier (tag) is generated for the sequence to be encoded, which corresponds to a binary fraction, which becomes the binary code for the sequence
- ✓ Thus, an average fractional bit rate (in bits per symbol) can be achieved without the need for blocking the symbols into compound symbols.

Arithmetic Coding

34

- ❑ It does not create tree and separate code for each symbol like Huffman rather it create a code named as tag for entire message
- ❑ Create an interval for each symbol, based on cumulative probabilities
- ❑ The interval for a symbol is [low, high).
- ❑ Consider we have a source having three symbols a_1 , a_2 , a_3 having probabilities of 0.4, 0.5, and 0.1 respectively. Generate a tag for the following sequence, a_2 , a_2 , a_2 , a_3 .
- ❑ Process starts with the unit subinterval $[0, 1)$

Symbol	Low	High
a_1	0	0.4
a_2	0.4	0.9
a_3	0.9	1

Arithmetic Coding

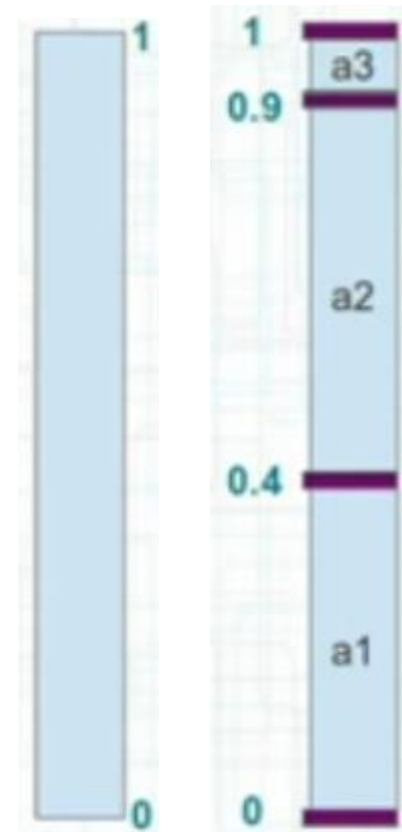
35

□ Arithmetic Coding-Steps

- ✓ Create a unit line

A2	A2	A2	a3
----	----	----	----
- ✓ Divide this line according to symbol and their probabilities
- ✓ The divided part is known as chunk

Symbol	Low	High
a1	0	0.4
a2	0.4	0.9
a3	0.9	1

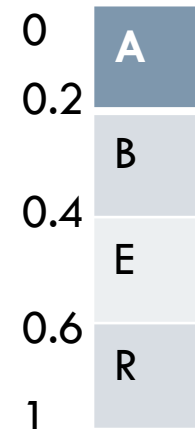


Arithmetic Coding

27

- Consider the string ARBER
- The intervals of symbols A, B, E, and R having probabilities of 0.2, 0.2, 0.2 and 0.4 respectively:

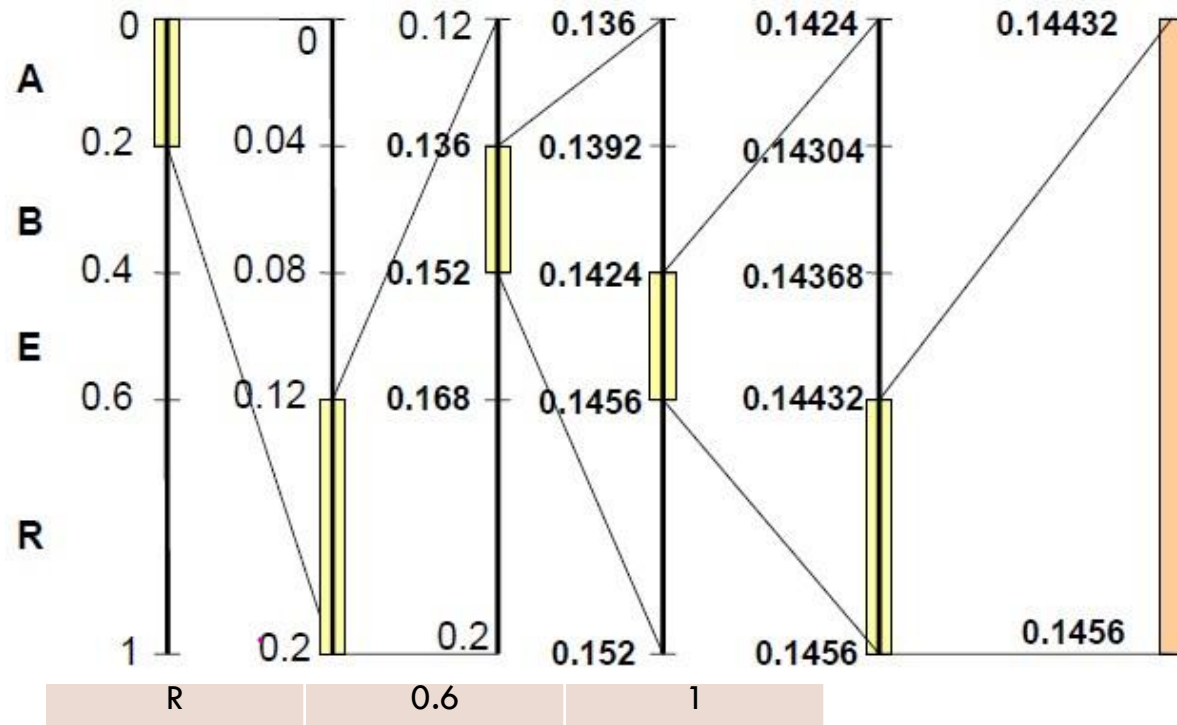
Symbol	Low Range	High Range
Initial	0	1
A	0	0.2
B	0.2	0.4
E	0.4	0.6
R	0.6	1



- A: $[0, 0.2)$; B: $[0.2, 0.4)$; E: $[0.4, 0.6)$; and R: $[0.6, 1)$;

Arithmetic Coding

28



- ❑ The final interval for the input string ARBER is [0.14432, 0.1456].
- ❑ In bits, one chooses a number in the interval and encodes the decimal part.

Arithmetic Coding

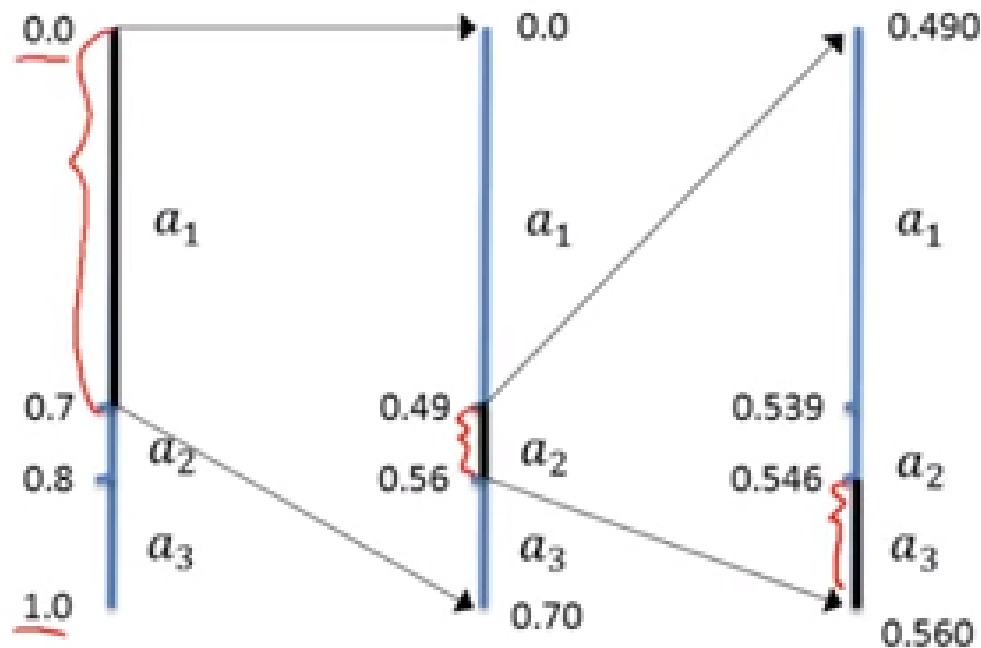
38

- ❑ Decipher the tag 0.55 to predict the source sequence for three symbols $\{a_1, a_2, a_3\}$, where $P\{a_1\}=0.7$, $P\{a_2\}=0.1$, $P\{a_3\}=0.2$.

Arithmetic Coding

39

- Decipher the tag 0.55 to predict the source sequence for three symbols $\{a_1, a_2, a_3\}$, where $P\{a_1\}=0.7$, $P\{a_2\}=0.1$, $P\{a_3\}=0.2$.



References

40

1. DIP by Gonzalez

For any query Feel Free to ask



41

